

Clock tree synthesis with TritonCTS

What it does Builds a buffered network that delivers a clock signal to many flip-flops with controlled delay, skew and signal transition time.

Remember this Divide clock sinks into groups, build balanced branches, and insert suitable clock buffers.

Why one clock wire is not enough

A clock can drive thousands of flip-flops. One driver connected to every clock pin sees a large electrical load. Long wires also add resistance and capacitance. Clock edges become slow, and distant sinks may receive them at different times.

Think of a school announcement system: local amplifiers distribute the same bell signal to groups of classrooms.

Four words you should know

Sink A receiving clock pin, such as a flip-flop clock input.

Buffer A cell that strengthens and redistributes the clock signal.

Clock latency The travel time from the clock reference to a sink.

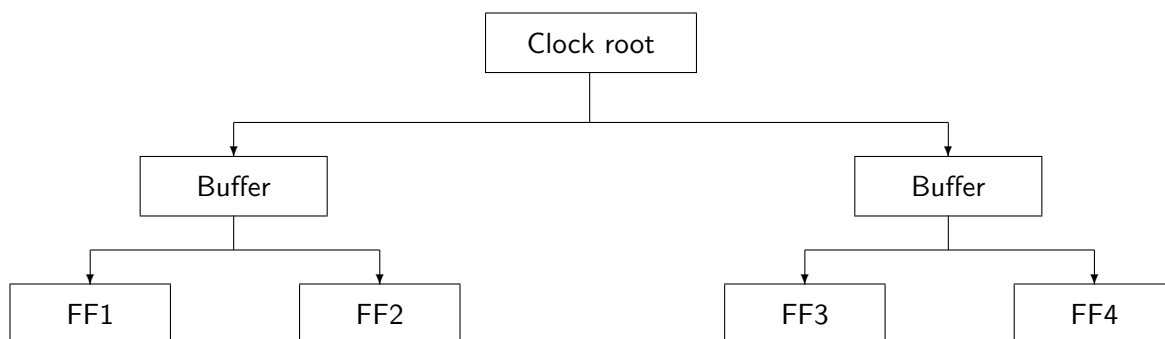
Clock skew The difference in clock arrival times between sinks.

The main algorithmic ideas

OpenROAD uses **TritonCTS 2.0**. It characterizes candidate buffer and wire segments using the loaded technology data, then uses this information while building the tree.

Clustering groups sinks spatially. Capacitated K-means, called **CKMeans** in the code, supports controlled partition sizes while building branches. An **H-tree based hierarchy** distributes the clock toward sink groups. Optional sink pre-clustering adds local buffered groups before that tree is built.

A simplified buffered clock tree

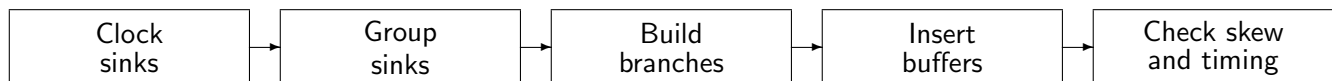


FF means flip-flop. This diagram shows connectivity, not an exact physical H shape. Real trees adapt to sink positions, loads, macros and constraints.

Clock tree synthesis with TritonCTS

The steps from input to output

- 1 **Gather sinks.** Read clock definitions, sink positions, buffers and wire RC data.
- 2 **Form branches.** Partition sinks and construct an H-tree based hierarchy.
- 3 **Choose buffers.** Use characterization to control capacitance and slew, meaning the time taken by a signal edge to transition.
- 4 **Balance and place.** Build buffered subnets and balance loads or latencies where supported and enabled.
- 5 **Recheck timing.** Legalize placement, update parasitics, propagate clock delays, and check setup and hold.



A small skew example

Before balancing, suppose the clock arrives at four sinks at:

0.10, 0.18, 0.25, 0.30 ns

The overall arrival spread is $0.30 - 0.10 = 0.20$ ns.

After CTS, suppose arrivals are:

0.22, 0.23, 0.24, 0.23 ns

The spread is now $0.24 - 0.22 = 0.02$ ns. Some sinks became later, but delivery became much more balanced. These are illustrative numbers, not a promised CTS result.

How clocks affect WNS and TNS

For a register-to-register path, both the launching and capturing clock arrival times affect slack. A later capture clock can help setup but hurt hold. CTS therefore needs timing checks after the actual clock network is modeled.

Equal wire length alone does not guarantee equal delay.

Tradeoff More buffers can improve drive strength and balance, but add area, clock power and insertion delay. CTS aims for acceptable skew and electrical limits, not necessarily zero skew.

Commands and a short spoken explanation

```
clock_tree_synthesis    report_cts
```

The platform must supply valid clock buffers and RC settings. `report_cts` reports tree counts; timing reports are needed to assess clock arrival times and slack.

“TritonCTS groups clock sinks, builds an H-tree based branching network, and inserts buffers using technology delay information. The aim is a well-driven clock with controlled skew, followed by setup and hold analysis.”

Sources and code OpenROAD Clock Tree Synthesis documentation. Code: `src/cts/src/HTreeBuilder.cpp`, `Clustering.cpp`, `SinkClustering.cpp`, `TechChar.cpp`.